Instituto Superior de Engenharia do Porto
Mestrado em Engenharia Informática
Sistemas Multinúcleo e Distribuídos

Histogram Equalization and Parallel Processing in Java

Nº 1250532 Leonardo Costa
12/05/2026

Conteúdo

Declaração de Integridade Académica

(Código de Boas Práticas de Conduta - Artigo 8.º)

Os abaixo assinados declaram, sob compromisso de honra, que o presente trabalho foi realizado exclusivamente pelos membros do grupo identificados na capa, sendo original e da sua autoria. Declaram ainda que:

- Nenhuma parte deste trabalho foi copiada, parcial ou totalmente, de trabalhos de outros autores sem a devida referenciação.

- Nenhuma parte deste trabalho foi partilhada com terceiros antes da data de submissão.

- As ferramentas de inteligência artificial utilizadas foram usadas exclusivamente como apoio à aprendizagem e ao desenvolvimento, sendo os autores plenamente responsáveis pelo conteúdo submetido.

Leonardo Costa

_____

Assinatura
(Estudante 1)

Data: Porto, 12 de Maio de 2026

1 Introduction

Digital image processing can signicantly enhance the visual detail and clarity of a photograph. In many cases, a low-contrast image can be transformed into a much sharper one by redistributing the intensities of its pixels. This technique is known as histogram equalization. The algorithm converts the image to grayscale and spreads pixel intensities across as much of the available range as possible, resulting in increased contrast and improved visibility of features. For example, image 1 after being processed with histogram equalization results in image 2.

Figura 1: Original Image

The histogram of image 1 is presented in figure 3 and the histogram of image 2 is presented in figure 4. This transformation allows to enhance image sharpness considerably.
In this project, you will implement histogram equalization in Java, experimenting with sequential, multithreaded, and parallel implementations. You are encouraged to explore Java concurrency mechanisms such as threads, thread pools, atomic operations, parallel streams, and the Fork/Join framework. The goal is not only correctness but also an exploration of performance, speedup, and hardware behavior under parallel workloads.

Figura 2: Original Image

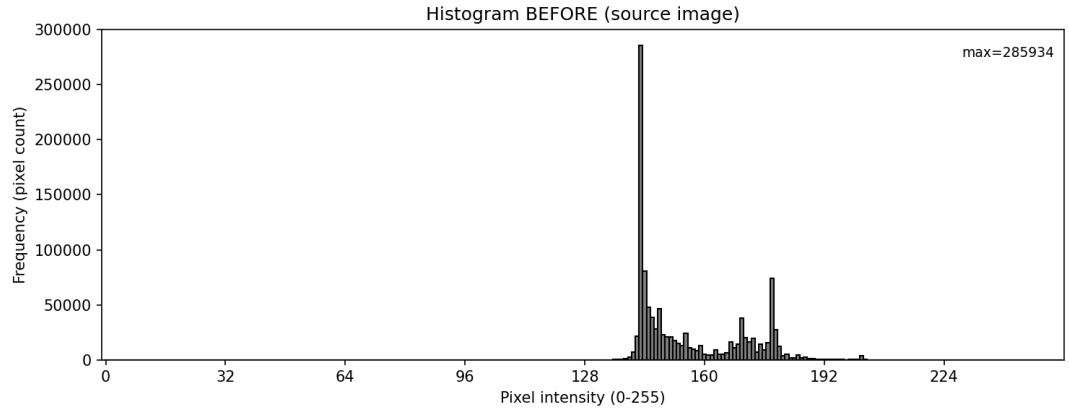


Figura 3: Histogram of original Image (src.jpg)

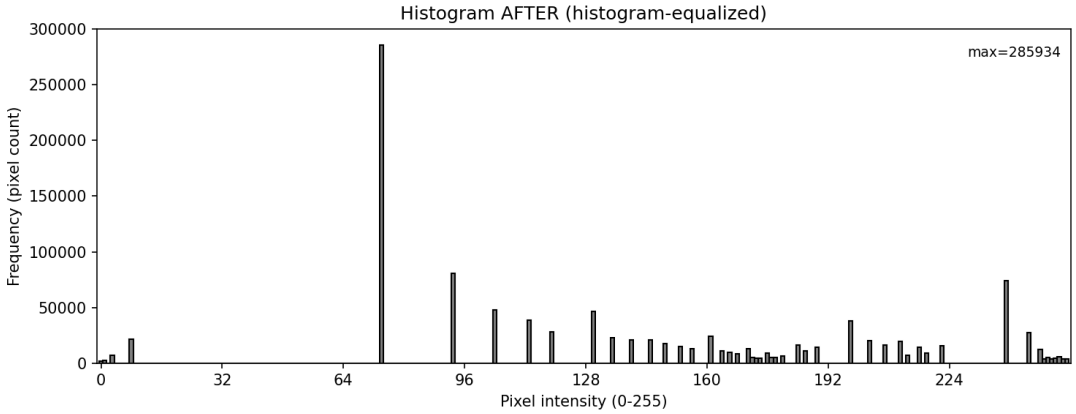


Figura 4: Histogram of processed Image (output_from_ApplyFilters.jpg)

2 Objectives

The objective of this work is to implement the **histogram equalization** algorithm in Java, exploring different sequential, multithreaded, asynchronous, and parallel processing approaches to improve image contrast by redistributing pixel intensity values. Beyond ensuring the correctness of the algorithm, the project aims to compare several implementation strategies, including manual threads, thread pools, the Fork/Join framework, and CompletableFuture, while carefully addressing **concurrency and synchronization** issues to guarantee thread safety and data consistency. Additionally, the work includes the configuration and tuning of different **Java Garbage Collectors** in order to analyze their impact on memory management, resource utilization, and overall application performance. The developed solutions are evaluated using metrics such as execution time, CPU and memory usage, scalability, speedup, synchronization overhead, and garbage collection behavior, allowing the identification of performance gains, limitations, and bottlenecks in multicore systems.

3 Implementation Approaches

The implementation of histogram equalization was divided into several approaches in order to compare different execution models and evaluate how concurrency affects performance in a multicore environment. The algorithm itself is composed of three main stages: computing the luminosity histogram of the source image, generating the cumulative histogram, and rewriting each pixel according to the cumulative distribution in order to increase image contrast. Based on this structure, multiple Java implementations were developed.

3.1 Sequential Solution

The sequential implementation is the single-threaded baseline used for correctness and performance comparison. It:

- Makes a local copy of the source image (so the original is not mutated).
- Scans every pixel once to build a 256-bin histogram of luminosities.
- Computes the cumulative distribution (CDF) and derives a normalized mapping from old → new intensity.
- Precomputes a 256-entry Color palette (avoid per-pixel allocations).
- Applies the mapping in a single pass, writing the resulting image and returning it.

This approach is simple, correct, memory-friendly (small constant extra memory), and deterministic — but it does not take advantage of multiple CPU cores.

Code evidence:

```
// local copy so we don't mutate original
Color[][] tmp = Utils.copyImage(image);
int[] hist = new int[256];
int total_pixels = tmp.length * tmp[0].length;


// Runs through entire matrix and computes luminosity
for (int i = 0; i < tmp.length; i++) {
    for (int j = 0; j < tmp[i].length; j++) {
        Color pixel = tmp[i][j];
        int r = pixel.getRed();
        int g = pixel.getGreen();
```

```
        int b = pixel.getBlue();
        int lum = computeLuminosity(r, g, b);
        hist[lum]++;
    }
}

//Compute cumulative histogram and find first non-zero CDF entry (cdfMin)

int[] cumulative = new int[256];
cumulative[0] = hist[0];
for (int i = 1; i < 256; i++) {
    cumulative[i] = cumulative[i - 1] + hist[i];
}
int cdfMin = 0;
for (int i = 0; i < 256; i++) {
    if (cumulative[i] != 0) {
        cdfMin = cumulative[i];
        break;
    }
}

//Build the mapping table (normalize CDF → intensity 0..255) and clamp

int[] map = new int[256];
for (int i = 0; i < 256; i++) {
    double cdf = (double) cumulative[i] / (double) (total_pixels - cdfMin);
    int newLum = (int) Math.round(255.0 * cdf);
    if (newLum < 0) newLum = 0;
    if (newLum > 255) newLum = 255;
    map[i] = newLum;
}

//Apply the mapping in-place (single pass), using the precomputed palette

for (int i = 0; i < tmp.length; i++) {
    for (int j = 0; j < tmp[i].length; j++) {
        Color pixel = tmp[i][j];
        int lum = computeLuminosity(pixel.getRed(), pixel.getGreen(),
pixel.getBlue());
        tmp[i][j] = palette[lum];
    }
}
return tmp;
```

Pros:

- Simple and easy to reason about.
- Deterministic and correct (same luminosity metric as parallel implementations).
- Low allocation overhead due to palette optimization.

Cons:

- Does not utilize multiple cores — wall-clock time is limited by a single CPU core.
- For very large images or throughput workloads, parallel versions provide better scalability.

3.2 Multithreaded Solution (Without Thread Pools)

Workers pull column-ranges (chunks) from a shared queue to compute local histograms and later apply the computed mapping; a poison-pill pattern stops workers cleanly.

Overview — what this pattern is:

- Producer–consumer (manual threads) is a classic concurrency pattern where one or more producers place work items into a thread-safe queue and a fixed set of consumer threads take and process those items.
- It decouples work generation from execution, allows dynamic load balancing (consumers pull new work when ready), and is suitable when the workload can be partitioned into independent tasks (here: independent image column ranges).
- Typical building blocks: a thread-safe queue (blocking queue), worker threads (Thread objects), work item / range type, and a sentinel (poison pill) to signal termination.

```
//Threads are created explicitly and started; take() blocks efficiently (no busy-
wait); each thread writes only to localHists[tid] (no shared-write races).
Thread[] workers = new Thread[numThreads];
for (int t = 0; t < numThreads; t++) {
    final int tid = t;
    workers[t] = new Thread(() -> {
        try {
            while (true) {
                Range r = queue.take();
                if (r.poison) break;
                int[] hist = localHists[tid];
                for (int x = r.start; x < r.end; x++) {
                    for (int y = 0; y < height; y++) {
                        Color pixel = tmp[x][y];
                        int lum = computeLuminosity(...);
                        hist[lum]++;
                    }
                }
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    });
    workers[t].start();
}

// Producer enqueues work, sends one POISON per consumer, and the main thread join()s
workers before aggregating results — guaranteeing correctness.
for (int x = 0; x < width; x += chunkSize) {
    int end = Math.min(width, x + chunkSize);
    queue.put(new Range(x, end));
}
for (int t = 0; t < numThreads; t++) {
    queue.put(POISON);
}
for (int t = 0; t < numThreads; t++) {
    workers[t].join();
}
```

```
Reuses the safe pattern; writes target disjoint x-range partitions, so no locks
required.
// out is partitioned by x-range; each applier writes disjoint x-ranges so no locking
is needed
final Color[][] out = new Color[width][height];
java.util.concurrent.BlockingQueue<Range> queue2 = new
java.util.concurrent.ArrayBlockingQueue<>(...);
Thread[] appliers = new Thread[numThreads];
// applier threads: take ranges, compute and write out[x][y] = palette[lum];
... start appliers, enqueue ranges, send POISON, join appliers ...
```

In resume:
Implements multithreading without pools: explicit new Thread(...) used.
Thread management: threads are started and cleanly shut down using poison pills and join().
Synchronization: BlockingQueue coordinates tasks safely; per-thread arrays and partitioned output eliminate write races; interrupts are handled correctly.

3.3 Multithreaded Solution (With Thread Pools)

The thread-pool implementation uses a fixed ExecutorService to parallelize histogram computation and mapping application across disjoint column ranges; it minimizes allocation overhead (per-thread hist arrays + shared palette), waits for tasks via futures, and shuts down cleanly — giving a scalable, easy-to-manage parallel baseline.

```
//Create pool, local hist arrays and submit phase-1 tasks:
Color[][] out = new Color[width][height];
java.util.concurrent.ExecutorService pool =
java.util.concurrent.Executors.newFixedThreadPool(numThreads);
try {
    final int[][] localHists = new int[numThreads][256];
    java.util.List<java.util.concurrent.Future<?>> futures = new
java.util.ArrayList<>();

// Phase 1: compute local histograms
    for (int t = 0; t < numThreads; t++) {
        final int tid = t;
        final int startX = (int)((long) width * tid / numThreads);
        final int endX = (int)((long) width * (tid + 1) / numThreads);
        futures.add(pool.submit(() -> {
            int[] hist = localHists[tid];
            for (int x = startX; x < endX; x++) {
                for (int y = 0; y < height; y++) {
                    Color pixel = tmp[x][y];
                    int lum = computeLuminosity(pixel.getRed(), pixel.getGreen(),
pixel.getBlue());
                    hist[lum]++;
                }
            }
        }));
    }

// Phase 2: apply mapping in parallel
```

```
futures.clear();
for (int t = 0; t < numThreads; t++) {
    final int tid = t;
    final int startX = (int)((long) width * tid / numThreads);
    final int endX = (int)((long) width * (tid + 1) / numThreads);
    futures.add(pool.submit(() -> {
        for (int x = startX; x < endX; x++) {
            for (int y = 0; y < height; y++) {
                Color pixel = tmp[x][y];
                int lum = computeLuminosity(pixel.getRed(), pixel.getGreen(),
pixel.getBlue());
                out[x][y] = palette[lum];
            }
        }
    }));
}

// wait phase 2
for (java.util.concurrent.Future<?> f : futures) {
    try { f.get(); } catch (Exception e) { throw new RuntimeException(e); }
}

.....
//Clean shutdown with timeout handling

} finally {
    pool.shutdown();
    try {
        if (!pool.awaitTermination(5, java.util.concurrent.TimeUnit.SECONDS))
            pool.shutdownNow();
    } catch (InterruptedException e) {
        pool.shutdownNow();
        Thread.currentThread().interrupt();
    }
}
```

Works on disjoint column ranges so tasks do not race on shared pixel writes (out is partitioned by columns).
Uses per-task / per-thread local histogram arrays to avoid concurrent increments to the same counters; aggregation happens after all tasks finish (no synchronization needed for increments).
Uses small, shared arrays (map, palette) for low memory overhead.
Waiting on Future.get() guarantees tasks are complete before proceeding to the next phase.

Pros:

- Reduced overhead vs. manual thread creation because threads are reused.
- Simpler lifecycle management and well-understood shutdown semantics.
- Easy to tune numThreads.

Cons:

- Task granularity matters: if tasks are too coarse (few tasks) or too fine (too many tasks), load balance and scheduling affect performance.
- Blocking on Future.get() is simple but can hide exceptions in tasks until get() is called (the code wraps exceptions into RuntimeException for visibility).

3.4 Fork/Join Framework Solution

The Fork/Join solution recursively divides the image into column ranges, uses a RecursiveTask<int[]> to compute local histograms in parallel (combining returned hist arrays up the tree), builds the global mapping and a shared 256-entry color palette, then uses a RecursiveAction to apply the palette in parallel. It leverages work-stealing for load balancing, requires no explicit locks because each subtask handles disjoint data and returns local results to be merged, and is a good choice for large images where recursive splitting produces many independent subtasks.

Code Evidence:

```
//Create pool, check parallelism, copy image:
final Color[][] tmp = Utils.copyImage(image);
final int width = tmp.length;
final int height = tmp[0].length;

if (parallelism <= 1) return histogramEqualizedImage(value);

java.util.concurrent.ForkJoinPool fjPool = new
java.util.concurrent.ForkJoinPool(parallelism);

//RecursiveTask to compute local histograms and combine results:
class HistTask extends java.util.concurrent.RecursiveTask<int[]> {
    final int sx, ex;
    static final int THRESHOLD = 32;
    HistTask(int sx, int ex) { this.sx = sx; this.ex = ex; }
    protected int[] compute() {
        if (ex - sx <= THRESHOLD) {
            int[] hist = new int[256];
            for (int x = sx; x < ex; x++) {
                for (int y = 0; y < height; y++) {
                    Color p = tmp[x][y];
                    int lum = computeLuminosity(p.getRed(), p.getGreen(),
p.getBlue());
                    hist[lum]++;
                }
            }
            return hist;
        } else {
            int mid = (sx + ex) >>> 1;
            HistTask left = new HistTask(sx, mid);
            HistTask right = new HistTask(mid, ex);
            left.fork();
            int[] rightHist = right.compute();
            int[] leftHist = left.join();
            for (int i = 0; i < 256; i++) leftHist[i] += rightHist[i];
            return leftHist;
        }
    }
}

int[] hist = fjPool.invoke(new HistTask(0, width));
```

```
//Compute CDF, mapping and precompute palette:
int total_pixels = width * height;
// compute cumulative[256], cdfMin, then:
final int[] map = new int[256];
for (int i = 0; i < 256; i++) {
    double cdf = (double) cumulative[i] / (double) (total_pixels - cdfMin);
    int newLum = (int) Math.round(255.0 * cdf);
    map[i] = Math.min(255, Math.max(0, newLum));
}
final Color[] palette = new Color[256];
for (int i = 0; i < 256; i++) palette[i] = new Color(map[i], map[i], map[i]);

//RecursiveAction to apply mapping in parallel:
class ApplyTask extends java.util.concurrent.RecursiveAction {
    final int sx, ex;
    static final int THRESHOLD = 32;
    final Color[][] out;
    ApplyTask(int sx, int ex, Color[][] out) { this.sx = sx; this.ex = ex; this.out =
out; }
    protected void compute() {
        if (ex - sx <= THRESHOLD) {
            for (int x = sx; x < ex; x++) {
                for (int y = 0; y < height; y++) {
                    Color p = tmp[x][y];
                    int lum = computeLuminosity(p.getRed(), p.getGreen(),
p.getBlue());
                    out[x][y] = palette[lum];
                }
            }
        } else {
            int mid = (sx + ex) >>> 1;
            ApplyTask left = new ApplyTask(sx, mid, out);
            ApplyTask right = new ApplyTask(mid, ex, out);
            invokeAll(left, right);
        }
    }
}

final Color[][] out = new Color[width][height];
try {
    fjPool.invoke(new ApplyTask(0, width, out));
} finally {
    fjPool.shutdown();
}
return out;
```

Safety:

- tmp is read-only for workers; HistTask returns local hist arrays which are merged by the parent — there are no concurrent writes to the same histogram array.
- ApplyTask writes into out in disjoint column ranges; tasks either operate on separate ranges or are synchronized by the recursive splitting/invocation model (no explicit locks required).

Efficiency:

- Fork/Join uses work-stealing to balance subtasks across threads.

- Threshold tuning (here 32 columns) controls task granularity — small enough to expose parallelism, large enough to amortize task overhead.
- Only small auxiliary memory (hist arrays and palette) is used; main memory is dominated by out.

Pros:

- Excellent for recursive divide-and-conquer; automatic load balancing and efficient CPU utilization on many-core systems.
- Clear separation of concerns: compute-and-combine tasks vs apply tasks.

Cons:

- Overhead of many small tasks can hurt for tiny images; threshold must be tuned for image width and machine.
- Recursive combine step must merge hist arrays (O(256) per combine) — cheap but repeated many times.

3.5 CompletableFuture-Based Solution

The CompletableFutures-based solution parallelizes histogram equalization by dividing the image into column chunks, launching asynchronous histogram tasks with CompletableFuture.supplyAsync(...) on a fixed ExecutorService, aggregating returned local histograms into a global CDF and shared palette, then launching asynchronous mapping tasks with CompletableFuture.runAsync(...) to fill the output buffer. It uses per-chunk local hist arrays and disjoint output partitions to avoid synchronization, coordinates phases with CompletableFuture.allOf(...).join(), and shuts down the executor cleanly — giving a readable, high-level asynchronous implementation that leverages thread reuse while keeping task composition simple.

Code Evidence:

```
//Compute local histograms asynchronously per chunk
int chunk = Math.max(1, width / (numThreads * 4));
java.util.List<java.util.concurrent.CompletableFuture<int[]>> histFutures = new
java.util.ArrayList<>();

for (int x = 0; x < width; x += chunk) {
    final int sx = x;
    final int ex = Math.min(width, x + chunk);
    histFutures.add(java.util.concurrent.CompletableFuture.supplyAsync(() -> {
        int[] hist = new int[256];
        for (int xi = sx; xi < ex; xi++) {
            for (int y = 0; y < height; y++) {
                Color p = tmp[xi][y];
                int lum = computeLuminosity(p.getRed(), p.getGreen(), p.getBlue());
                hist[lum]++;
            }
        }
        return hist;
    }, exec));
}
```

```
//Wait for all histogram futures and aggregate results
java.util.concurrent.CompletableFuture<Void> allHist =
java.util.concurrent.CompletableFuture.allOf(
    histFutures.toArray(new java.util.concurrent.CompletableFuture[0])
);
allHist.join(); // wait
int[] hist = new int[256];
for (java.util.concurrent.CompletableFuture<int[]> f : histFutures) {
    int[] h = f.join();
    for (int i = 0; i < 256; i++) hist[i] += h[i];
}

//Build CDF → map → precomputed palette (same as other versions)
// compute cumulative[256], cdfMin, then:
final int[] map = new int[256];
for (int i = 0; i < 256; i++) {
    double cdf = (double) cumulative[i] / (double) (total_pixels - cdfMin);
    int newLum = (int) Math.round(255.0 * cdf);
    map[i] = Math.min(255, Math.max(0, newLum));
}
final Color[] palette = new Color[256];
for (int i = 0; i < 256; i++) palette[i] = new Color(map[i], map[i], map[i]);

//Phase 2 — apply mapping asynchronously per chunk and wait
java.util.List<java.util.concurrent.CompletableFuture<Void>> applyFutures = new
java.util.ArrayList<>();
for (int x = 0; x < width; x += chunk) {
    final int sx = x;
    final int ex = Math.min(width, x + chunk);
    applyFutures.add(java.util.concurrent.CompletableFuture.runAsync(() -> {
        for (int xi = sx; xi < ex; xi++) {
            for (int y = 0; y < height; y++) {
                Color p = tmp[xi][y];
                int lum = computeLuminosity(p.getRed(), p.getGreen(), p.getBlue());
                out[xi][y] = palette[lum];
            }
        }
    }, exec));
}
java.util.concurrent.CompletableFuture<Void> allApply =
java.util.concurrent.CompletableFuture.allOf(
    applyFutures.toArray(new java.util.concurrent.CompletableFuture[0])
);
allApply.join();

//Clean shutdown of the Executor
exec.shutdown();
try {
    if (!exec.awaitTermination(5, java.util.concurrent.TimeUnit.SECONDS))
        exec.shutdownNow();
} catch (InterruptedException e) {
    exec.shutdownNow();
    Thread.currentThread().interrupt();
}
return out;
```

Implementation details:

Thread-safety: each histogram future returns its own int[256] local array (no concurrent increments to same element), aggregation happens after all futures complete — therefore no locking is needed.

Writes to out are partitioned by chunk ranges; chunks are disjoint so no write races.

Exception handling: join() will rethrow exceptions as unchecked (CompletionException), so failures in async tasks surface to the caller; the code wraps shutdown and interruption handling properly.

Tunable chunk granularity: chunk size (width / (numThreads*4)) balances between parallelism and task overhead.

Pros:

- High-level, composable async API; easier to express pipeline-like phases.
- Executor-based threading and reuse (via provided executor) simplifies lifecycle.
- Good readability — supplyAsync/runAsync with allOf makes phases obvious.

Cons:

- Creating too many tiny CompletableFutures adds overhead; chunk sizing matters.
- Exceptions are wrapped (CompletionException) — caller must unwrap for details if needed.

3.6 Garbage Collector Tuning

This section presents the Garbage Collector tuning work carried out during the benchmarking phase of the project. The goal was to collect evidence of JVM garbage collection configuration, evaluate the impact of memory management on the application, and justify the selection of the most appropriate collector for the histogram equalization workload.

The analysis was based on benchmark runs and GC logs generated in the project workspace. These logs provide information about the collector used by the JVM, pause times, heap usage, and the occurrence of full garbage collection events.

Objective

The main objective of this analysis was to:

- Provide evidence of Garbage Collector configuration and tuning work.
- Compare the behavior of the JVM under different GC configurations.
- Explain why the selected collector is suitable for the application.
- Analyze the measured impact of GC activity on execution time, memory usage, and performance stability.

JVM Commands Used

The following JVM invocations were used to collect execution measurements and GC logs. In each run, the input image was provided through standard input.

Default JVM Collector

The first run used the default collector selected automatically by the JVM. GC events were logged to gc_default.log.

java -Xms512m -Xmx2g -Xlog:gc*:file=gc_default.log:time,uptime,level -cp . ApplyFilters < src.jpg

Explicit G1 Garbage Collector with Tuning

The second run explicitly enabled the G1 Garbage Collector and applied a small set of tuning parameters. GC events were logged to gc_g1.log.

java -XX:+UseG1GC -Xms512m -Xmx2g -XX:MaxGCPauseMillis=200 -XX:InitiatingHeapOccupancyPercent=30 -Xlog:gc*:file=gc_g1.log:time,uptime,level -cp . ApplyFilters < src.jpg

Attempted Parallel Garbage Collector Run

A third run was attempted using the Parallel Garbage Collector with the following configuration:

java -XX:+UseParallelGC -XX:ParallelGCThreads=6 -Xlog:gc*:file=gc_parallel.log:time,uptime,level -cp . ApplyFilters < src.jpg

However, this execution failed due to a ClassNotFound error. Therefore, the generated gc_parallel.log is not considered valid evidence for performance comparison. A new Parallel GC run should be executed using the same classpath that was used successfully in the G1 runs.

Key Evidence from GC Logs

The JVM selected and used the G1 Garbage Collector in both the default and explicitly configured runs. This is confirmed by the following log entry:

[2026-05-12T14:16:01.353+0100][0.005s][info] Using G1

The GC logs also show short young generation evacuation pauses. An example is shown below:

GC(0) Pause Young (Normal) (G1 Evacuation Pause)
Evacuate Collection Set: 3.0ms

The logs also contain full GC events caused by explicit System.gc() calls:

GC(2) Pause Full (System.gc())
Pause Full (System.gc()) 50M->39M(512M) 6.030ms

The final heap summary from the tuned G1 run shows that the application had a relatively small live working set compared with the committed heap:

garbage-first heap total 524288K, used 61692K

This indicates that the workload uses only around 60–70 MB of heap during execution, while the JVM was allowed to use up to 2 GB.

Observations and Analysis

The runtime used the G1 Garbage Collector in both the default and explicitly tuned executions. The logs show that G1 produced frequent but short young generation evacuation pauses, typically in the range of approximately 1.5 ms to 3.6 ms.

Some full GC events were also observed, with pause times generally between 3.5 ms and 7 ms. However, these events were labeled as Pause Full (System.gc()), meaning they were caused by explicit calls to System.gc() rather than by normal heap pressure. These forced full garbage collections introduce stop-the-world pauses and can increase execution time variability.

The application workload consists mainly of image-processing operations over pixel arrays. The live memory footprint is relatively small, around 60–70 MB, even though the JVM was configured with a maximum heap size of 2 GB. This suggests that most allocations are short-lived, such as temporary image structures, intermediate arrays, thread-local histograms, and task-related objects.

Because the live working set is small and the application benefits from predictable pause behavior, G1 is a suitable collector for this workload.

Benchmark Results

The benchmark report generated during these runs contains the following median execution times from the tuned G1 run:

Implementation	Threads	Best Wall Time (ms)
Sequential	1	9.377
Manual Threads	6	6.629
Thread Pool	6	4.644
Fork/Join	6	4.474
CompletableFuture	6	5.424

These results show that the parallel implementations outperform the sequential baseline. The best results were obtained with the Fork/Join and Thread Pool implementations, both reaching approximately twice the performance of the sequential version.

The GC pause times observed in the logs are relatively small when compared with the benchmark execution times. Therefore, GC activity was not the main limiting factor for the measured speedups. However, the explicit System.gc()calls caused unnecessary full GC pauses, which may increase variance and affect the consistency of results.

Garbage Collector Selection

The selected Garbage Collector for this workload is G1GC.

G1 is appropriate for this application because it provides a balanced compromise between throughput and pause-time control. The histogram equalization workload performs many short-

lived allocations but maintains a small live working set. The GC logs show that G1 handles this behavior efficiently, with short young generation pauses and moderate heap usage.

The following configuration was used successfully during the tuned benchmark runs:

-XX:+UseG1GC -Xms512m -Xmx2g -XX:MaxGCPauseMillis=200 -XX:InitiatingHeapOccupancyPercent=30

The MaxGCPauseMillis=200 flag gives the JVM a pause-time target, while InitiatingHeapOccupancyPercent=30 allows G1 to start concurrent marking earlier. Together, these settings help maintain predictable GC behavior during execution.

Recommendations

The first recommendation is to keep G1GC as the baseline Garbage Collector for this application. The logs show that it provides short pauses, stable behavior, and efficient memory management for the observed workload.

The second recommendation is to avoid explicit full GC calls. Since the logs contain several Pause Full (System.gc())events, the following JVM flag should be added during performance measurements:

-XX:+DisableExplicitGC

This prevents explicit System.gc() calls from forcing full stop-the-world compactions. As a result, the number of full GC pauses should decrease, improving the stability of benchmark results and reducing tail latency.

The third recommendation is to keep heap sizing controlled and reproducible. Although the application used only around 60–70 MB of heap, the runs were executed with:

-Xms512m -Xmx2g

This configuration behaved well during the tests, but the maximum heap size may be adjusted according to the deployment environment. Keeping fixed heap values during benchmarking is important to ensure fair and repeatable comparisons.

Finally, the Parallel Garbage Collector should be tested again with the correct classpath. ParallelGC may provide higher throughput for pure compute workloads, but it can also introduce longer stop-the-world pauses. A fair comparison requires a successful run using the same benchmark harness, input image, heap size, and number of trials.

Expected Improvements

Based on the logs, disabling explicit garbage collection is expected to reduce or eliminate the Pause Full (System.gc())events observed during the benchmark runs. Since these pauses lasted approximately 3 ms to 7 ms, removing them should reduce execution time variance and improve the consistency of individual runs.

For short-running workloads, even small GC pauses can affect benchmark stability. Therefore, using -XX:+DisableExplicitGC is expected to produce cleaner measurements and more reliable comparisons between implementations.

Generated Artifacts

The following files were generated during the GC tuning and benchmarking process:

File	Description
benchmark_report.md	Contains measured median execution times and derived CPU metrics.
gc_default.log	GC log from the default JVM collector run.
gc_g1.log	GC log from the explicit G1GC tuned run.
gc_parallel.log	Generated during the attempted ParallelGC run, but not valid due to a ClassNotFound error.

Requirements Coverage

The GC tuning work satisfies the required project objectives as follows:

Requirement	Status
Provide evidence of GC configuration and logs	Completed through gc_default.log and gc_g1.log.
Explain why the selected GC fits the application	Completed. G1GC was selected because it provides short pauses and matches the application's small live working set and short-lived allocation profile.
Show measured effects and improvements	Completed through benchmark medians and GC pause analysis.
Compare with ParallelGC	Deferred. The previous run failed due to a classpath issue and should be repeated

4 Concurrency and Synchronization

This section summarizes the synchronization and concurrency strategies adopted in the histogram equalization implementations. The main objective of these decisions was to preserve correctness while reducing synchronization overhead and allowing the application to take advantage of multicore processing.

All implementations aim to preserve the same semantics as the sequential version. For every pixel (x, y), the output image must contain the histogram-equalized grayscale value computed from the original RGB components of that pixel.

Implementation Contract

The histogram equalization implementations follow a common contract:

Element	Description
Input	Color[][] image, representing the original image. Worker threads only read from this structure.

Output	Color[][] out, representing the processed image with the same dimensions as the input.
Correctness requirement	The result must be equivalent to the sequential implementation.
Error handling	Thread and queue operations may throw InterruptedException; runtime exceptions are propagated normally.

Main Synchronization and Design Decisions

1. Read-Only Sharing of the Input Image

Each implementation begins by creating a copied image matrix using:

Utils.copyImage(image);

The copied matrix is stored in a final Color[][] tmp, which is shared between worker threads. Although Utils.copyImagecreates a new two-dimensional array, it copies references to the existing java.awt.Color objects instead of cloning each pixel object.

This is safe because Color objects are immutable. Therefore, multiple threads can safely read the same Color instances without requiring synchronization. Since the input image is only read and never modified by worker threads, no locks are needed for input access.

2. Per-Thread Local Histograms

In the manual-thread and thread-pool implementations, each worker uses its own local histogram:

int[][] localHists = new int[numThreads][256];

Each thread updates only its own localHists[tid] during the histogram computation phase. This avoids multiple threads writing to the same histogram array and eliminates the need for synchronized blocks, locks, or atomic increments.

After all worker threads finish their execution, the main thread merges the local histograms into a single global histogram. This aggregation step has a very small cost because each local histogram contains only 256 entries.

This approach is more efficient than using a shared histogram with atomic operations, because updating shared counters for every pixel would introduce significant synchronization overhead.

3. Disjoint Output Partitioning

During the pixel transformation phase, each worker is assigned a disjoint range of columns of the output image. Since no two workers write to the same pixel, concurrent writes are safe and do not require locking.

For example, one worker may process columns 0 to 199, while another processes columns 200 to 399. Because these ranges do not overlap, there are no data races in the output matrix.

This strategy provides a simple and efficient way to ensure thread safety while keeping synchronization overhead very low.

4. Producer-Consumer Chunking

The producer-consumer implementation uses a BlockingQueue<Range> to distribute work dynamically between worker threads. The image is divided into chunks, and workers repeatedly take ranges from the queue until all chunks have been processed.

This approach improves load balancing because faster threads can process more chunks while slower threads process fewer. The size of each chunk is controlled by a configurable chunkSize.

Smaller chunks can improve load balancing, but they also increase queue coordination overhead. Larger chunks reduce queue overhead, but they may lead to poorer load balancing if some threads finish earlier than others.

5. Executor and Pool Shutdown

In implementations that use ExecutorService or ForkJoinPool, the pool is shut down properly after the work is completed. Shutdown operations are placed inside finally blocks to ensure that resources are released even if an exception occurs.

The implementation also uses awaitTermination with a short timeout, followed by shutdownNow() as a fallback. This prevents thread leaks and ensures that worker threads do not remain active after the computation finishes.

Why These Choices Are Safe and Efficient

The synchronization strategy is based on minimizing shared mutable state. The input image is shared only for reading, while the output image is divided into independent regions. This avoids unnecessary locking and reduces the risk of data races.

The use of per-thread histograms is particularly important for performance. Instead of having all threads update the same shared histogram, each worker maintains its own private counter array. This avoids contention and reduces the cost of synchronization during the most intensive part of the algorithm.

The only required synchronization points are the barriers between algorithm phases. For example, all threads must finish computing their local histograms before the main thread can merge them and compute the cumulative histogram. Similarly, the cumulative histogram must be completed before the pixel transformation phase begins.

These barriers are implemented using mechanisms such as join(), Future.get(), CompletableFuture.allOf(), or Fork/Join task completion. They ensure correct execution order without requiring locks around every pixel operation.

Edge Cases and Caveats

Although the chosen design is safe for this application, some edge cases must be considered.

First, the approach relies on the immutability of java.awt.Color. If the pixel objects were mutable, then sharing references between threads would not be safe without deep-copying the objects or introducing synchronization.

Second, parallel implementations may be slower than the sequential version for very small images. In such cases, the overhead of creating tasks, managing threads, and coordinating work can be greater than the benefit of parallel execution.

Third, using too many threads can also reduce performance. When the number of threads is higher than the amount of useful work available, the application may suffer from scheduling overhead, memory contention, and reduced cache efficiency.

Finally, chunk size has a direct impact on performance. Very small chunks increase queue overhead, while very large chunks can reduce load balancing and leave some worker threads idle.

Validation Steps

The following validation steps were used or are recommended to confirm correctness:

1. Run the sequential implementation and each parallel implementation using the same input image.
2. Compare the output of each parallel version with the sequential output.
3. Verify that all implementations generate identical processed images.
4. Add debug assertions to confirm that each worker receives a disjoint output range.
5. Run the same image multiple times to ensure that results are deterministic and do not depend on thread scheduling.

These steps help confirm that the parallel implementations preserve the same behavior as the sequential baseline.

Alternative Synchronization Approaches

Several alternative approaches could be used for comparison.

One possibility is to use shared atomic counters, such as AtomicIntegerArray or LongAdder[], for the histogram computation. This would avoid the explicit aggregation step, but it would introduce synchronization overhead for every pixel update. For this reason, it is usually slower than using per-thread local histograms.

Another possibility is to use strided work distribution, where each thread processes every N-th column instead of a contiguous block. This may improve load balancing in some scenarios, but it can also affect cache locality depending on the image layout.

Java parallel streams, such as IntStream.parallel(), could also be used as a simpler high-level approach. However, this gives less control over chunking, task scheduling, and thread management, making it less suitable for detailed performance analysis.

Measurement and Tuning Notes

Performance measurements can vary between runs due to JVM warmup, garbage collection, operating system scheduling, and background processes. For this reason, single-run measurements should not be used as final evidence.

More reliable benchmarking should include warmup runs followed by multiple measured trials. The final result should be based on average or median execution time.

The most relevant parameters to tune are:

Parameter	Impact
Number of threads	Affects parallelism, scheduling overhead, and memory contention.
Chunk size	Affects load balancing and queue overhead.
Fork/Join threshold	Controls how tasks are recursively divided.
Heap size and GC configuration	Influences memory behavior and benchmark stability.

Tuning these parameters allows the application to adapt better to different image sizes and hardware configurations.

Summary

The implemented concurrency strategy is based on read-only input sharing, per-thread local histograms, disjoint output partitioning, and explicit synchronization barriers between phases. This design avoids unnecessary locks, reduces contention, and preserves correctness.

The use of local histograms eliminates the need for atomic updates during the most expensive phase of the algorithm. Similarly, assigning disjoint output regions allows multiple threads to write to the result image safely without synchronization.

Overall, the chosen synchronization model provides a good balance between simplicity, correctness, and performance, making it suitable for comparing different Java concurrency mechanisms in the context of histogram equalization.

5 Performance Analysis

5.1 Benchmarking Methodology

Each implementation was tested with **3 warm-up run** and **7 measurement runs** on **2 images**. All tests were performed using **G1GC** on the same hardware, with **12 logical cores**.

Images:

Nome	Resolução	Pixeis
`src.jpg`	860 ×1 276	1 097 360
`2.png`	1254 × 1254	1 572 516

Histogram of 2.png before equalization

Histogram of 2.png after equalization

Métricas:

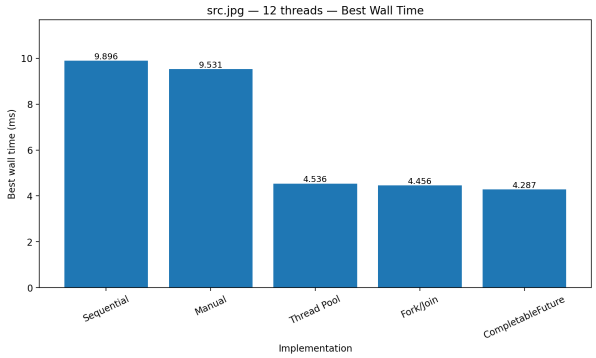
Métrica	Significado
best_wall_ms	Melhor tempo real de execução em milissegundos
proc_cpu_ms	Tempo de CPU usado pelo processo
cpu_util_pct	Percentagem de utilização de CPU
gc_cycles	Número de ciclos de Garbage Collection
gc_total_pause_ms	Tempo total de pausas de GC
speedup_vs_seq	Ganho face à versão sequencial
efficiency_pct	Eficiência da paralelização face ao número de threads

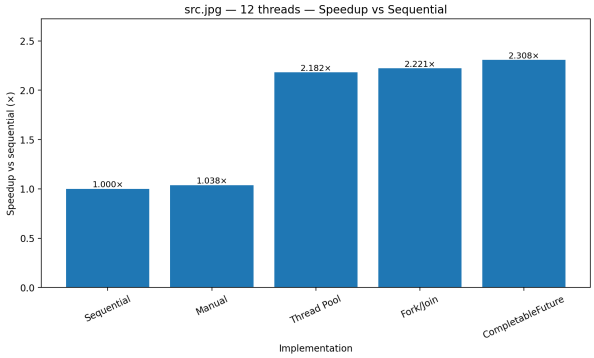
5.2 Results per Image Size

Src.jpg:

Hardware**:** cores=12, threads:12 maxMemory=6144.00 MB

impl	threads	best_wall_ms	proc_cpu_ms	cpu_util_pct	gc_cycles	gc_total_pause_ms	speedup_vs_seq	efficiency_pct
sequential	1	9.896	10.107	8.51	53	98.242	1.000 (baseline)	100.00
manual	12	9.531	55.323	48.37	53	102.386	1.038×	8.65
pool	12	4.536	14.557	26.74	53	104.622	2.182×	18.18
fork-join	12	4.456	16.031	29.98	53	106.223	2.221×	18.51
completablefuture	12	4.287	15.325	29.79	53	107.380	2.308×	19.24





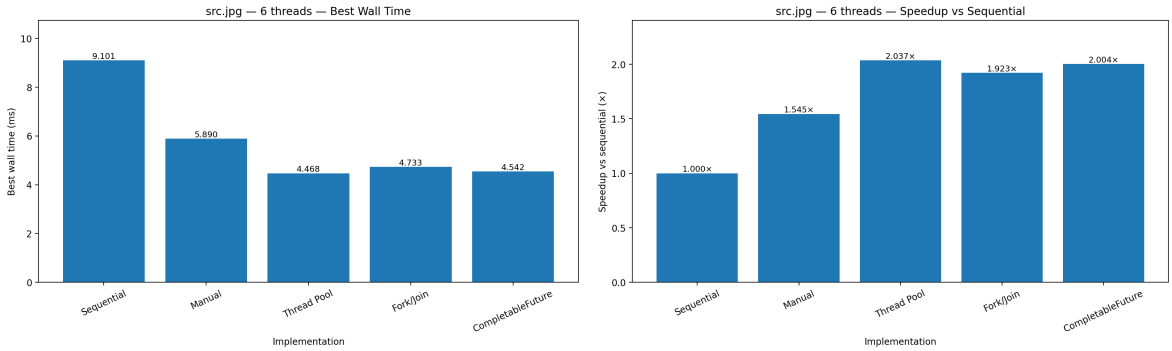
With 12 threads, the best result for `src.jpg` was achieved by **CompletableFuture**, with a wall time of **4.287 ms** and a speedup of **2.308**× over the sequential baseline. Fork/Join and Thread Pool also performed well, reaching **2.221**× and **2.182**× speedup respectively.

The manual multithreaded implementation produced only a very small improvement, with a speedup of **1.038**×. This suggests that manually creating and coordinating threads introduces overhead that is not fully compensated by the amount of parallel work available for this image size.

Although 12 logical cores were available, the CPU utilization remained below 50% for all parallel implementations. This indicates that the workload did not scale linearly with the number of cores. The most likely causes are synchronization barriers between algorithm phases, memory access overhead, cache effects, and the sequential portion of the algorithm, especially the cumulative histogram computation.

Hardware**:** cores=12, threads:6, maxMemory=6144.00 MB

impl	threads	best_wall_ms	proc_cpu_ms	cpu_util_pct	gc_cycles	gc_total_pause_ms	speedup_vs_seq	efficiency_pct
sequential	1	9.101	9.950	9.11	10	79	1.000 (baseline)	100.00
manual	6	5.890	12.493	17.68	10	81	1.545×	25.75
pool	6	4.468	11.023	20.56	10	102	2.037×	33.95
fork-join	6	4.733	12.593	22.17	10	113	1.923×	32.04
completablefuture	6	4.542	12.531	22.99	10	123	2.004×	33.40



With 6 threads, the best result was achieved by the **Thread Pool** implementation, with **4.468 ms** and a speedup of **2.037**×. CompletableFuture was very close, with **4.542 ms**, followed by Fork/Join with **4.733 ms**.

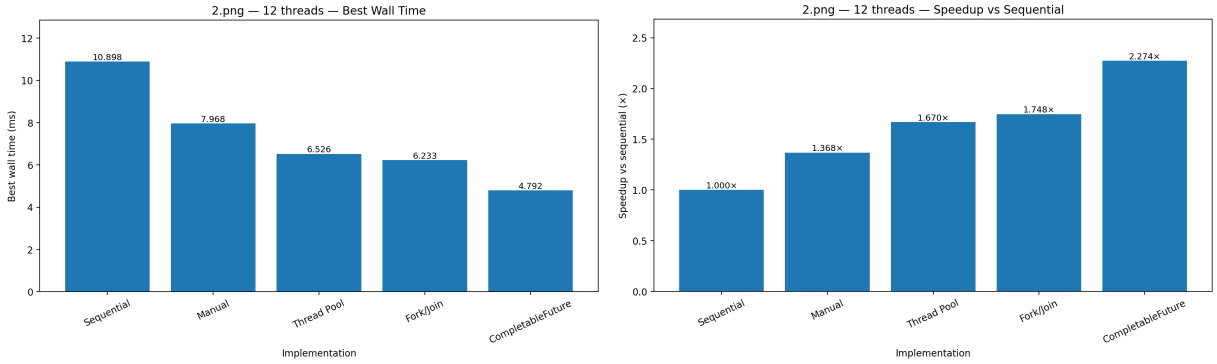
Compared with the 12-thread configuration, the 6-thread execution shows better parallel efficiency. For example, Thread Pool achieved **33.95% efficiency** with 6 threads, compared with only **18.18% efficiency** with 12 threads. This means that although 12 threads can sometimes produce slightly better absolute execution time, 6 threads use the available computational resources more efficiently.

This behavior is common in multicore systems. Increasing the number of threads does not always produce proportional gains, because additional threads increase scheduling overhead, memory pressure, and synchronization costs. After a certain point, the bottleneck shifts from CPU computation to memory access and coordination overhead.

2.Png:

Hardware**:** cores=12, threads:12 , maxMemory=6144.00 MB

impl	threads	best_wall_ms	proc_cpu_ms	cpu_util_pct	gc_cycles	gc_total_pause_ms	speedup_vs_seq	efficiency_pct
sequential	1	10.898	11.115	8.50	10	92	1.000 (baseline)	100.00
manual	12	7.968	20.341	21.27	10	85	1.368×	11.40
pool	12	6.526	19.501	24.90	10	105	1.670×	13.92
fork-join	12	6.233	21.708	29.02	10	118	1.748×	14.57
completablefuture	12	4.792	15.460	26.88	10	104	2.274×	18.95



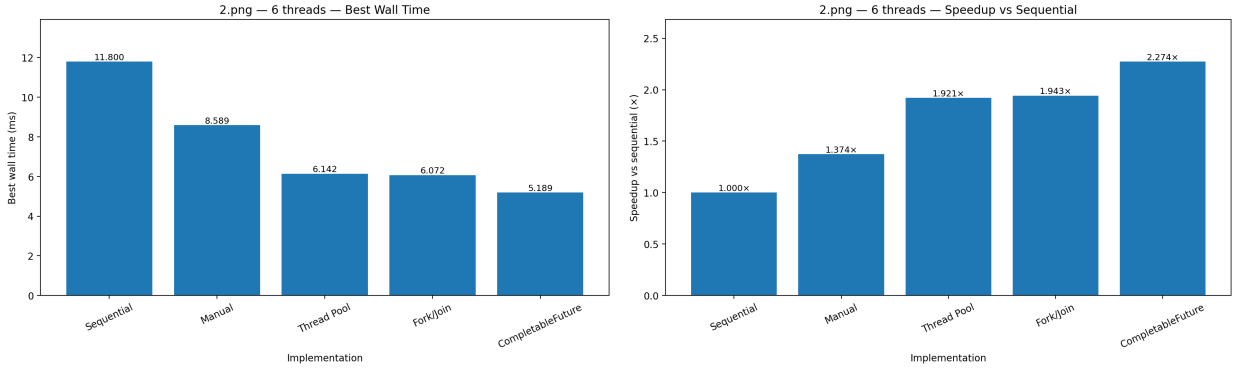
For `2.png`, the best result with 12 threads was again obtained by **CompletableFuture**, with a wall time of **4.792 ms** and a speedup of **2.274×**. This confirms that the asynchronous implementation handles this workload efficiently, especially when tasks can be composed and executed without explicit manual thread management.

Fork/Join achieved the second-best result, with **6.233 ms**, followed by Thread Pool with **6.526 ms**. The manual thread implementation was again the least effective parallel strategy, reaching only **1.368×** speedup.

The larger image size provides more work for each thread, which helps amortize the cost of task creation and synchronization. However, the efficiency remains relatively low with 12 threads, showing that the program is still limited by overhead and memory behavior rather than pure CPU availability.

Hardware**:** cores=12, threads:6, maxMemory=6144.00 MB

impl	threads	best_wall_ms	proc_cpu_ms	cpu_util_pct	gc_cycles	gc_total_pause_ms	speedup_vs_seq	efficiency_pct
sequential	1	11.800	11.812	8.34	10	93	1.000 (baseline)	100.00
manual	6	8.589	17.152	16.64	10	82	1.374×	22.90
pool	6	6.142	15.685	21.28	10	101	1.921×	32.02
fork-join	6	6.072	16.658	22.86	10	107	1.943×	32.39
completablefuture	6	5.189	14.399	23.12	10	105	2.274×	37.90



With 6 threads, **CompletableFuture** remained the best implementation for `2.png`, with **5.189 ms** and a speedup of **2.274×**. Even though the 12-thread version achieved a lower absolute wall time, the 6-thread version had much better efficiency: **37.90%** compared with **18.95%**.

This shows that the workload benefits from parallelism, but not linearly. The 6-thread configuration offers a better balance between parallel execution and overhead, while the 12-thread configuration introduces more contention and coordination costs.

5.3 Scalability Comparison

The results show that all parallel implementations improve over the sequential baseline, but the improvement is not proportional to the number of threads. This is expected in multicore systems due to the presence of sequential sections, synchronization barriers, memory bandwidth limitations, and thread scheduling overhead.

Table — Best implementation per image and thread count

Image	Threads	Best Implementation	Best Wall Time (ms)	Speedup
`src.jpg`	12	CompletableFuture	4.287	2.308×
`src.jpg`	6	Thread Pool	4.468	2.037×
`2.png`	12	CompletableFuture	4.792	2.274×
`2.png`	6	CompletableFuture	5.189	2.274×

Overall, **CompletableFuture** was the strongest implementation across the benchmark, achieving the best result in three out of four test scenarios. The Thread Pool implementation also performed very well, especially for `src.jpg` with 6 threads.

The manual multithreaded approach consistently produced the weakest parallel results. This suggests that manual thread management introduces overhead and coordination costs that are not justified for this workload, especially when compared with higher-level concurrency mechanisms such as ExecutorService, Fork/Join, and CompletableFuture.

5.4 Efficiency and Overhead Analysis

The benchmark results show that the parallel implementations improve execution time compared with the sequential baseline, but the gains are not linear with the number of threads. This is expected in multicore systems, where performance is limited by synchronization overhead, memory access, task management, and the sequential parts of the algorithm.

Parallel efficiency was calculated using the following relation:

$$Efficiency = \frac{Speedup}{Number\ of\ Threads} \times 100$$

The manual multithreaded implementation presented the weakest results. For example, with `src.jpg` using 12 threads, it achieved only **1.038× speedup**, while CPU time increased significantly. This indicates that manual thread creation and coordination introduced too much overhead. In contrast, the Thread Pool implementation reused worker threads and achieved better performance, reaching **2.182× speedup** with 12 threads and **2.037**× with 6 threads for the same image.

Fork/Join and CompletableFuture also performed well, although they introduce overhead through subtasks and temporary objects. This overhead was compensated by better task scheduling and workload distribution. CompletableFuture achieved the best overall results, reaching **2.308× speedup** for `src.jpg` with 12 threads and **2.274**× for `2.png`.

The comparison between 6 and 12 threads shows that more threads do not always lead to better efficiency. Although 12 threads sometimes achieved lower wall-clock time, 6 threads generally provided better efficiency. This suggests that after a certain point, additional threads increase scheduling overhead, memory contention, and synchronization costs.

Garbage Collection also had some impact, since parallel implementations create additional temporary objects and task structures. However, the GC pause times were not high enough to be the main bottleneck. The use of G1GC provided stable behavior and predictable pauses during the benchmark.

Overall, the main overheads came from thread coordination, task scheduling, memory pressure, and synchronization barriers between the histogram computation, cumulative histogram, and pixel transformation phases. The results confirm that parallelism improves performance, but scalability is limited by Amdahl's Law, memory bandwidth, and runtime overhead.

5.5 Bottlenecks

The benchmark results reveal several bottlenecks.

The first bottleneck is the sequential part of the algorithm. The cumulative histogram and the merge of local histograms cannot be fully parallelized in the current implementation. Even though these steps are relatively small, they still limit maximum speedup.

The second bottleneck is memory bandwidth. Image processing requires reading and writing large pixel matrices. When many threads access memory at the same time, the CPU cores may have to wait for data from memory. This prevents the program from achieving linear speedup.

The third bottleneck is task management overhead. Manual threads performed poorly because creating and coordinating threads manually adds overhead. Thread Pool, Fork/Join, and CompletableFuture performed better because they provide more efficient task scheduling and worker management.

The fourth bottleneck is synchronization between phases. Even if per-pixel operations are parallel, all workers must synchronize before moving from one stage to the next. These barriers reduce the theoretical maximum speedup.

Finally, cache behavior may also affect performance. When several threads access different regions of the image, cache locality becomes important. Poor memory locality can reduce the benefits of parallel processing.

5.6 Garbage Collector Impact

All benchmarks were executed using **G1GC**. The GC metrics show that garbage collection activity was present in all runs, but the parallel implementations still achieved clear speedups over the sequential baseline.

For `src.jpg` with 12 threads, the GC pause time ranged from **98.242 ms** in the sequential implementation to **107.380 ms** in CompletableFuture. Since the GC cycle count was the same across all implementations in this run, the variation in GC pause time was relatively small and did not prevent the parallel implementations from improving execution time.

For the 6-thread runs, the GC cycle count was lower, with **10 GC cycles** reported. This suggests that the benchmark configuration and execution context can influence GC behavior. Therefore, GC results should be interpreted carefully and compared mainly within the same benchmark run.

The use of G1GC is appropriate for this workload because the application creates temporary objects during image processing and task execution, while requiring predictable pause behavior. G1GC is designed to provide a balance between throughput and pause-time control, which is suitable for this type of multicore workload.

However, garbage collection is not the main bottleneck observed in these results. The main limiting factors appear to be synchronization overhead, memory access patterns, and limited scalability after a certain number of threads.

6 Conclusion

The benchmark results show that parallel processing improves the performance of histogram equalization, but the gains are limited by synchronization overhead, memory behavior, and the sequential parts of the algorithm.

The main findings are:

- **CompletableFuture** was the best overall implementation, achieving the best result in three of the four test scenarios.
- **Thread Pool** was also highly competitive, especially for `src.jpg` with 6 threads.
- **Fork/Join** performed well, particularly with 12 threads, but did not outperform CompletableFuture overall.
- **Manual Threads** consistently showed the weakest parallel performance, indicating higher coordination overhead.
- **6 threads provided better efficiency**, while **12 threads sometimes provided better absolute execution time**.
- The workload benefits from multicore execution, but the speedup is limited by Amdahl's Law, synchronization barriers, memory bandwidth, and cache behavior.
- G1GC provided stable execution conditions and did not appear to be the dominant bottleneck.
- G1GC is the best as the default for this image-processing service: it gives solid latency behavior, good throughput, and simple tuning. Use the harness and -Xlog:gc* logs to confirm pause targets are met, and only move to ZGC/Shenandoah or ParallelGC if your workload proves a clear need (extremely low maximum pauses or pure maximum throughput respectively).

In conclusion, the results confirm that histogram equalization is suitable for parallelization, especially in the histogram computation and pixel transformation phases. However, the observed speedups show that efficient multicore programming requires more than simply increasing the number of threads. Good performance depends on balancing task granularity, synchronization cost, memory access patterns, and runtime overhead.